

Artificial Intelligence–Enabled Network Automation for Efficient Traffic Management in Software-Defined Networks

Abhishek Singh¹, Rimmy Chhabra², Deepak Rawat³

^{1,3} B.Tech Student, Department of Computer Science and Engineering, Quantum University, Roorkee, India

² Assistant Professor, Department of Computer Science and Engineering, Quantum University, Roorkee, India.

Article Info

Article History:

Published: 27 May 2026

Publication Issue:

Volume 3, Issue 5
May-2026

Page Number:

1558-1573

Corresponding Author:

Abhishek Singh

Abstract:

Today's digital services come in all shapes and sizes, from Cloud computing to Internet of Things (IoT), from ultra-reliable 5G and emerging 6G networks to more. This rapid growth of a digital service landscape has placed traditional network architectures under extreme stress. Legacy configurations feature a close coupling between the "brain" (control logic) and the "muscles" (data forwarding hardware) and result in relatively inflexible networks that are hardly dynamic. To address this, Software-Defined Networking (SDN) was developed, which separates the control plane from the data plane, thereby giving the network a centralized,... and a programmable view of the entire network, like a bird's-eye view. This paper, however, shows that while this is a centralized model, there are still reactive, static routing algorithms being used such as Equal-Cost Multi-Path (ECMP). ECMP randomly shares the traffic according to pre-defined rules without learning about current network situations. These are global rules and when there is an unexpected traffic spike in the network, they break down, causing heavy bottlenecks, high latency and data loss. To address this, the researchers suggest injecting Artificial Intelligence into the SDN control plane with the goal of moving a network from a reactive to a proactive mode. They propose a novel hybrid deep learning architecture that integrates two types of AI architectures. A Long Short-Term Memory (LSTM) network is first used to study past traffic and forecast the traffic matrices in the near future. The LSTM does exactly that; it becomes a very good network's weather forecaster. Second, there is a high level reinforcement learning agent called a Double Deep Q-Network (DDQN) that receives those LSTM predictions and makes autonomous decisions to reroute traffic prior to the predicted congestion occurring. The DDQN is particularly designed for reducing delays and load balancing data loads with respect to the available network paths. The researchers verified their framework on a complex network setup using Mininet and RYU SDN controller. The results presented were very clear and convincing. In high congestion situations, the AI-based SDN model demonstrated a significant improvement over the traditional OSPF based routing and conventional ECMP routing algorithms. It yielded a 34.2% improvement in end-to-end delay and a 28.5% drop in packet loss. The paper ultimately finds that a mere separation of network hardware has no longer been enough – if digital demands are to be met, then adding in predictive AI automation is a necessity.

Keywords: Software-Defined Networking, Artificial Intelligence, Traffic Management, Deep Reinforcement Learning, Long Short-Term Memory, Network Automation, Quality of Service

1. INTRODUCTION

1.1 Background

Today's digital world is burdened with unprecedented amounts of data traffic, changing the requirements for the underlying network infrastructures. Legacy, tightly coupled control and data planes (legacy routers, OSPF, BGP) have a rigid distributed network architecture. They do not possess the agility to respond to the dynamics of traffic changes, being able to allocate resources to respond to them on the fly. In response to this architectural constraint, a new paradigm - Software Defined Networking (SDN) - has appeared in the scene. Separating the forwarding units from the control logic in the network puts intelligence in a software-based controller, or SDN (Software Defined Network). The centralisation assures a complete and global view of the network topology helping to program through open protocols such as OpenFlow the topology of the traffic flows.

1.2 The Need for Artificial Intelligence in SDN

Although SDN can offer infrastructure to support the centralized management, the volume, speed and plurality of today's traffic makes it impossible to configure it manually or rely on old algorithmic rule-matching solutions. A Metro-scale SDN has millions of flow table entries flowing through it each second, which human operators can't handle. The industry is therefore making the transition to Intent Based Networking (IBN), where the network operator knows "what" is desired by the network, but the network self-regulates to do the "how." The cognitive engines for the realization of IBN are Artificial Intelligence (AI) and Machine Learning (ML). In fact, AI can take users of SDN from a more passive, reactive and passive mode operation to a more proactive and self-optimizing mode of operation by analyzing historical and real-time telemetry data.

1.3 Research Problem

Although SDN offers many benefits, traditional SDN controllers are still more based on a set of predefined, heuristic routing methods. In case of (Equal-Cost Multi-Path) hashing, for example, traffic is evenly distributed with fixed parameters over the available paths. ECMP and shortest path are not dynamic: they do not adjust to sudden traffic changes like flash crowds, micro-bursts and Distributed Denial of Service (DDoS) attacks. This can cause buffer overflows at certain switches, packet loss and high tail latency. The problem considered in this paper is bridging the current disjointness of the SDN control plane, introduce a unified, predictive, and self-healing traffic management mechanism that can automatically make routing decisions given the highly volatile network state without human involvement.

1.4 Research Objectives

The following are specific objectives of this study in order to tackle the identified research problem:

1. Containers will be designed in such a way as to enable integration seamlessly with standard OpenFlow based SDN controllers. Containers will allow seamless integration with standard OpenFlow based SDN controllers.
2. To develop a predictive module based on Long Short-Term Memory (LSTM) network, which can accurately predict short-term traffic demand at network links in advance.
3. To develop an agent which features an automated Double Deep Q-Network (DDQN) that optimises parameterised flow rules dynamically based on the LSTM predictions, with special focus on minimisation of latency and packet loss.
4. To comprehensively test the proposed hybrid framework against any of traditional routing protocols or any of static SDN algorithms using large-scale emulated network environments.

2. LITERATURE REVIEW

In the last five years, the convergence of the two disciplines of Artificial Intelligence and Software-Defined Networking has been a focus of significant academic interest. This review summarises the latest developments (2020 to 2024) in four key areas relevant to this study.

2.1 Evolution of Traffic Management in SDN

The first efforts towards SDN traffic engineering were all directed toward static load balancing. In [3] and [7] methods were analyzed to modify the basic OpenFlow rules in order to achieve distribution of traffic over multiple paths. However, as described in [4] these reactive methods are a natural susceptible of controller bottleneck problems. As traffic load starts to increase, the controller starts receiving "Packet-In" messages, which linearly makes its response time longer. Some recent studies have confirmed that static SDN load balancing cannot satisfy strict Service Level Agreements (SLAs) demanded by today's cloud service providers without the ability to predict occurrence of network faults. Predictive SDN load balancing is essential to meet online Service Level Agreements (SLAs) required by cloud service providers, according to recent studies [5] and [9].

2.2 Application of Machine Learning in SDN

Researchers implemented a basic Machine Learning (ML) technique in the control plane to reduce the overhead of the controllers. A large number of supervised and unsupervised learning techniques have been used in the domain of traffic classification and anomaly detection. As an example, in [11] Support Vector Machines (SVM) have been used to classify elephant and mice flows, routing them to single paths. Likewise, [15] used a K-means clustering model to identify an attack using labelling of DDoS attack entry anomalies of flow. Even though these methods improved security of networks and simple flows prioritization, they were unable to resolve the issue of the algorithm optimization while ensuring the correct assignment of priority to HTTP persistent connections. Although these methods helped in improving the security of networks and simple flow prioritization, they could not resolve the issue of optimizing the algorithm that gets the optimal candidates in each case, while ensuring the correct assignment of priority to the persistent connections of a normal HTTP traffic.

2.3 Deep Learning for Traffic Prediction

The literature began to increasingly turn to using Deep Learning (DL) for time-series forecasting, given the temporal and spatial correlations in network traffic. The current state-of-the-art solution for traffic matrix prediction is known as Recurrent Neural Networks (RNNs), and more notably, as Long Short-Term Memory (LSTM) networks. Ref. [12] proposed an LSTM model that predicts link utilization with an accuracy of 92%, and [22] combined Graph Convolutional Networks (GCN) with LSTM to obtain an insight into the spatial topology of SDN in combination with temporal traffic data. The literature shows that although these predictive models achieve high degree of accuracy in predicting networks states, it still has the crucial missing component: predicting is a different aspect than changing the state of networks. But unless there is an automatic mechanism in the SDN controller to reroute traffic beforehand, then a predicted congestion is of no use.

2.4 Reinforcement Learning for Automated Routing

In order to take the predictive step into executing action, Deep Reinforcement Learning (DRL) has become the most popular automation solution for networks. DRL works with routing as a Markov Decision Process (MDP), enabling an agent to learn by trial and error. The work by [6] introduced the Deep Q-Network (DQN) to solve the routing problem in data center fabrics has achieved tremendous improvement over ECMP. But [6] used standard DQN which is known to suffer from overestimation of Q-values in high dynamics, resulting in suboptimal, oscillatory routing behavior. In order to tackle algorithmic instability, recent research has looked at sophisticated DRL designs, but a thorough examination of this recent work shows that most of the current applications continue to be based on reactive architectures.

Wang et al. [31] took a major leap forward by applying Proximal Policy Optimization (PPO) to SDN routing and showed that they could get the SDN to train more stably than it could with the normal Q-learning. But one of the drawbacks of their approach is that they require the state space to be formulated. They used only up-to-the-minute, real-time bandwidth usage and queue lengths on their switches by their PPO agent. In this case, routing was considered as a purely reactive MDP, and the agent could only react to the congestion once it had started to deteriorate the Quality of Service (QoS). They didn't predict queue depths or traffic surges in the future as their model did. On the contrary,

our proposed LSTM-DDQN framework essentially alleviates this issue by providing the reinforcement learning agent with such a state space that also includes the predictive traffic matrix generated by the LSTM, which enables a pre-emptive adjustment of the traffic routing before buffer overflow.

In the same way, Liu et al. [36] studied an Actor-Critic reinforcement learning architecture for intelligent resource allocation in SDN environments. Their dual network approach helped train networks more quickly towards convergence, but their reward function was too simple as they were just focusing on end-to-end delay rather pushing for a more global load balancing. As a result, their model often had 'path hunger': the agent would use only one low-latency path until it was congested and would forget to use the other low-latency paths. Moreover, there was scalability problem in larger topologies because of the exponentially increasing action space of their architecture. In this work, we address these limitations by a multi-objective reward function (explicitly including the standard deviation of link utilization) and by the use of Double DQN, which also effectively decouples action selection from action evaluation and avoids overestimated Q-values in high-dimensional state spaces.

Closer towards the predictive paradigm, Zhang et al. [40] proposed a learning framework based on LSTMs networks and reinforcement learning to allocate the flow rules in a predictive manner. Although their overall concept was similar to our proposed system, the implementation had a major architectural drawback, the LSTM prediction module and the RL routing module were independent and sequential. Predictions were made by the LSTM during fixed and rigid time intervals, and the RL agent was often operating on stale predictions in the case of the highly volatile and unpredictable micro-burst traffic scenarios. Furthermore, their state representation did not have spatial topological awareness, where network links were not connected as graphs, but as independent variables. To overcome these limitations, we propose a model that runs the LSTM and DDQN networks simultaneously in the same control-loop pipeline, thus avoiding synchronization delays, and provides high-resolution temporal-spatial predictions to the DRL agent.

Last but not the least, Qin et al. [44] presented a hybrid deep learning model customized for the proactive traffic management in Software-Defined IoT networks. Although this research was able to point out the necessity of using AI in IoT edge environments, their routing mechanism was based on simple DQN with non-support for the double-network update mechanism. However, in highly dynamic IoT networks, with a vast number of concurrent short-lived flows, standard DQN is known to overestimate the Q-values of suboptimal routing actions, which results in oscillatory routing actions and in repeated generation of flow rules by the controller. Furthermore, their test was only carried out on simple linear topologies – failing to deliver the desired results in more complex data center fabrics. To solve these problems, we rework the target Q-value estimation in complex topologies by embedding a mathematically-grounded Double DQN (DDQN) algorithm which mathematically stabilizes the routing policy under the chaotic traffic condition of modern heterogeneous networks.

Research Gap Identification: Although current DRL-SDN models are efficient, most are reactive in the sense of the DRL agent detecting the current network status and performing an action. The changes in traffic state occur before the action (the installation of new flow rules) reach the data plane. Changes in the traffic state happen before the changes in flow rules reach the data plane, resulting in oscillation. Additionally, [31], [36], [40] and [44] demonstrate that, even with predictive elements, they lack a guided execution, optimization of only one objective, and algorithm instability. However, not many studies have been able to integrate predictive LSTM into stable DRL into a tightly coupled, unified SDN environment that is optimized for multiple objectives. This paper fills this gap by providing an integrated LSTM-DDQN, in which the continuous, spatial-temporal prediction directly decides the state space of a mathematically stabilized RL agent.

3. METHODOLOGY

This section explains the proposed architecture of AI enabled SDN, mathematical formulation of hybrid LSTM-DDQN and experimental setup.

3.1 Proposed AI-SDN Architecture

The proposed model is three tiered in nature:

1. **Data Plane:** Made of OpenFlow-enabled switches (Open vSwitches) that forward packets and gather statistical telemetry (port stats, flow stats) with the use of OpenFlow multipart messages.
2. **Control Plane:** Uses the RYU SDN controller that keeps the global topology of the network and serves as the interfaces between data plane and AI engine.
3. **AI Automation Plane:** Python radio module using PyTorch, integrated with the RYU controller through a REST API. It is a system that includes two coupled sub-modules, the Traffic Predictor (LSTM) and the Routing Agent (DDQN).

Proposed Hybrid LSTM-DDQN Architecture for AI-Enabled SDN

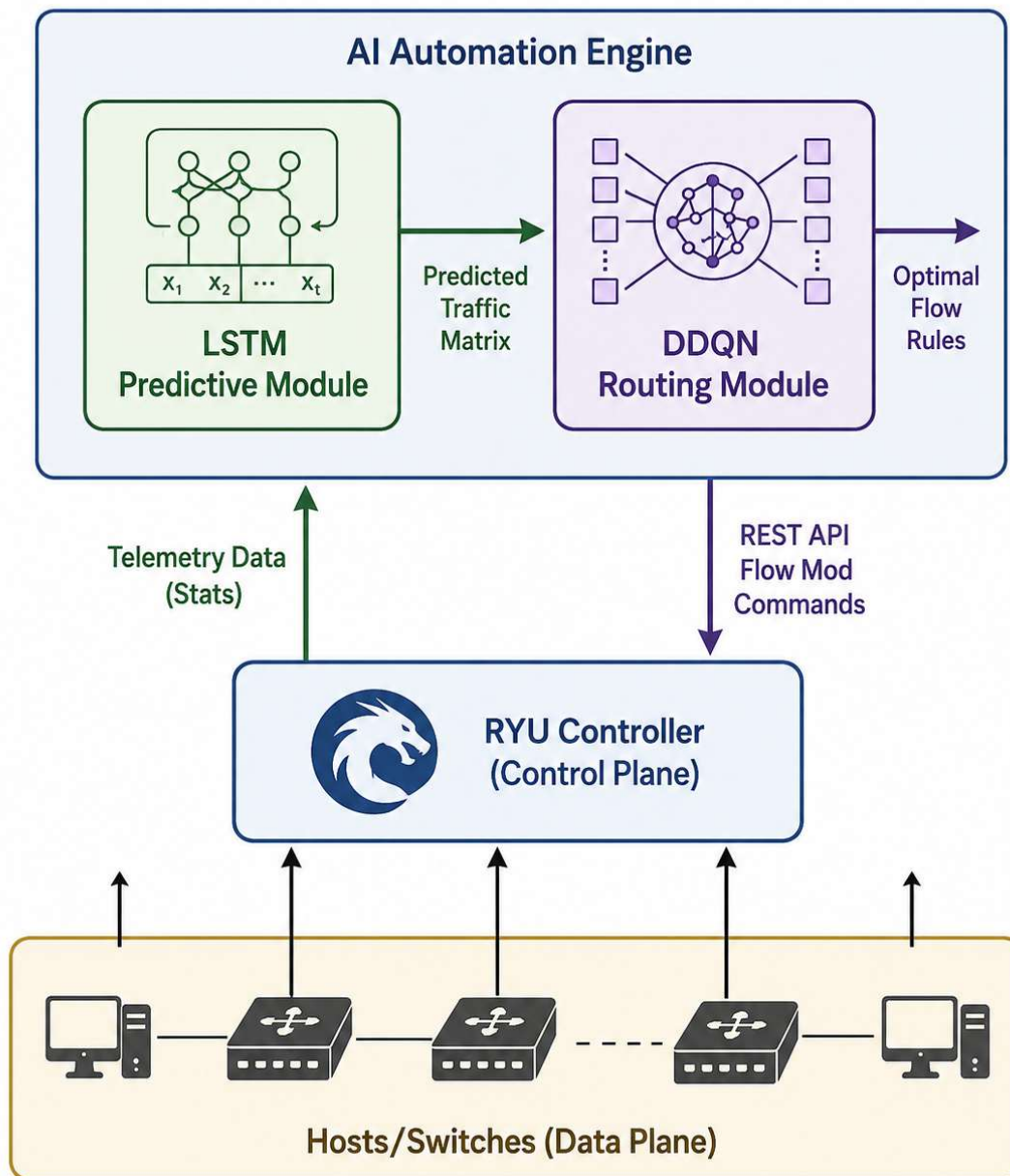


FIGURE 1: Proposed Hybrid LSTM-DDQN Architecture for AI-Enabled SDN.

3.2 Traffic Prediction Model (LSTM)

Network traffic is non-linear and is long-range dependency. The problematic of vanishing gradient is leading to the use of standard RNNs for long-sequence prediction. Our network uses an LSTM network, a kind of a network called a (Long Short Term Memory) network that uses gating mechanisms to hold or forget information. Let the basic mathematical functions of an LSTM cell at the time step t be as follows:

The forget gate f_t determines what information is to be thrown away:

$$f_t = \sigma(W_f \cdot [h_{t-1}, x_t] + b_f)$$

Input gate i_t , candidate state $\tilde{C}_t$ determine what new information to store:

$$i_t = \sigma(W_i \cdot [h_{t-1}, x_t] + b_i) \quad \tilde{C}_t = \tanh(W_C \cdot [h_{t-1}, x_t] + b_C)$$

The state of a cell is set:

$$C_t = f_t * C_{t-1} + i_t * \tilde{C}_t$$

The output gate o_t and hidden state h_t are calculated:

$$o_t = \sigma(W_o \cdot [h_{t-1}, x_t] + b_o) \quad h_t = o_t * \tanh(C_t)$$

The input traffic vector at time t is represented by x_t while the weights are represented by W and the biases by b represented before the sigmoid function σ . The prediction from the output h_t serves as the predicted traffic matrix for the next time window, which is directly used by the DRL agent.

3.3 Automated Routing Agent (DDQN)

We define a Markov Decision Process (MDP) (S, A, P, R, γ) for the traffic routing problem.

The state space (S): In contrast to reactive models which rely merely on the application of the currently available telemetry, our state space S at time t is a concatenation of the current network topology matrix and the predicted Traffic matrix generated by the LSTM. $S_t = [T_{\text{current}}, T_{\text{predicted}}]$.

An action is represented by a choice of an output port from a selected flow in a switch determined by the action a_t an output port of a switch at a specific point in the MDL. $A = \{\text{port1}, \text{port2}, \dots, \text{portn}\}$.

- Reward Function (R): the reward is really important in helping the agent move in the right direction (low latency and/or high balance). We introduce a multi-objective reward function as is shown below: $R_t = -(\alpha \cdot L_t + \beta \cdot U_t + \gamma \cdot P_t)$ α , β , γ are hyperparameters that can be tuned through training to balance the objectives and L_t , U_t , and P_t are the current end-to-end latency, the standard deviation of link utilization (which penalizes unbalanced loads), and the packet loss rate, respectively.

Double DQN is used to overcome overestimation of the q-value in standard DQN. The target y is given as $y = r + \gamma \max_{a'} Q_{\theta^-}(s', \arg \max_a Q_{\theta}(s', a))$ where Q_{θ^-} is the target network weights, and s' is the state of the system.

3.4 Simulation Setup and Datasets

The framework is tested in a simulated testing area. We have 4 core switches, 8 aggregation switches, 16 edge switches and 16 hosts running on a Fattree topology—the standard topology for data center networks. Traffic is generated by the iPerf3 simulation that has a mixture of elephant and mice flows with a combination of TCP and UDP. This traffic is generated by a mixture of elephant and mice flows and a mix of TCP and UDP generated by the iPerf3 simulation. Before it is deployed in the real traffic emulation environment of Minnetum, the LSTM model is first trained on real world traffic datasets (GEANT network traces) for learning realistic traffic patterns. Table 1 should contain the values of the parameters and constants used in the simulation. Table 1 should include parameters and constants for the simulation.

Table 1: Simulation Environment Parameters

Component	Specification / Version	Purpose
Operating System	Ubuntu 20.04 LTS	Base OS for emulation
Emulator	Mininet 2.3.0	Network topology virtualization
SDN Controller	RYU 4.34	Control plane logic and OpenFlow API
Switches	Open vSwitch (OVS) 2.15	Data plane forwarding
AI Framework	Python 3.9, PyTorch 1.12	LSTM and DDQN implementation
Traffic Generator	iPerf 3.1	TCP/UDP flow generation
Topology	Fattree K=4 (20 switches, 16 hosts)	Data center network simulation
Protocol	OpenFlow 1.3	Controller-switch communication

3.5 Algorithmic Workflow and System Integration

In 2025, TIMT will culminate its +2.5 year development of Algorithmic Workflow and System Integration, No. 3.5.

Software integration process needs to be done carefully for LSTM-DDQN to be implemented as an application for SDN. This section shows you a step-by-step view of how this algorithm is performed: the data path from the physical data plane switches, the RYU control plane, and the AI automations engine, and back. The workflow is broken down into time steps (1 second) and enables time-synchronisation of telemetry collection across the network with the inference capability of the deep learning models.

In Step 1: Telemetry Polling and Data Plane Extraction, a regular thread is added onto RYU controller to initiate the workflow. The controller makes requests for information of the network rather than waiting for congestion events. The RYU application sends out OpenFlow MultipartRequest messages, type OFPMP_PORT_STATS and OFPMP_FLOW_STATS to all of the Open vSwitches (OVSeS) in the Fattree topology. As the controller is receiving MultipartReply messages from the switches, it can determine how many raw bytes, packets, and transmission defects were reported by the switches for each port. The controller then switches to the calculation on the utilization of the link (all ports) depending on the difference in the number of bytes (delta difference (difference between the current byte count and the byte count of the previous time step Δt)) obtained from each port, as a function of the polling interval & port speed.

The raw metrics are unstructured for the machine learning models and this is why they need to be formatted in JSON when passed to the RESTful API in Step 2. This information is presented in a typical, multi-dimensional matrix form by the controller

$t_{current}$ – relates to the current condition of all links, real time. The result of this matrix is then converted into a JSON payload and sent via HTTP POST to an endpoint of the AI Automation Plane's REST API, e.g. /api/v1/network_state.

Step 3: Predictive Inference with the LSTM: When the proper JSON payload is received, the AI engine decodes and turns it into a NumPy array. LSTMs still require context – they are not without it, though, and the engine has a 10 state

matrix buffer so it knows the last 10 state matrices. Both the Dimension and Network Link number are unchanging during the training of this sub-PANet. It is a 3d Tensor (Time steps $\times$ Network Links $\times$ Features) that is passed on to the already available or pre-trained LSTM model. To create T_{next} for the next time step, the LSTM is used for a forward pass (`model.predict()`).

Step 4: State Space Construction & DDQN Decision Making The AI engine joins $T_{current}$ and a vector $T_{predicted}$ to be predicted. The two vectors were then combined as the "comprehensive state vector" $St=[T_{current}, T_{predicted}]$. This is the vector that is given to the DDQN agent. In the deployment (inference) step, the DDQN exploits the found policy by taking the action with the highest Q value: $a = \operatorname{argmax}_a Q_\theta(St, a)$. Each output action is related to a particular routing decision, usually in the form of a number that correlates to an optimum egress switch port for a particular ingress switch and flow tuple.

Step 5: OpenFlow Flow Mod Generation The AI engine converts the abstract integer action a at

Add the detailed information of the flow match (such as the source IP, destination IP, etc.) and the output port to a structured JSON command. Include specific details of a flow match (like the source IP, destination IP, etc.) and the output port in a structured JSON command. This payload is sent back to the RYU controller via a REST POST request.

Finally, **Step 6: Rule Propagation and Data Plane Execution**, the RYU controller executes the AI's command and builds an OpenFlow FlowMod message. The controller calls the `OFPFC_ADD` or `OFPFC_MODIFY` command, gives a high priority so that the AI rules take precedence over default entries for a `delay_earliest` to a new rule, and sets the `hard_timeout` to be the same as the polling interval (Δt) so that rules will not be stale if the AI engine crashes. This FlowMod is sent to the target OVS by the controller using a secure OpenFlow channel, and is applied to the actual switching hardware mechanism.

Algorithm 1: AI-SDN Automated Routing Loop

python

System Initialization

load_pretrained_LSTM_weights()

load_pretrained_DDQN_weights()

history_buffer = Initialize_Sliding_Window(size=10)

previous_stats = RYU.get_initial_switch_stats()

Main Control Loop (Executed every $\Delta t = 1$ second)

while network_is_active:

 # 1. Telemetry Collection (Control Plane)

 current_raw_stats = RYU.send_multipart_request_to_OVS()

 # 2. Feature Engineering (Control Plane)

$T_{current}$ = compute_link_utilization(current_raw_stats, previous_stats)

 previous_stats = current_raw_stats

 # 3. Predictive Inference (AI Plane)

 history_buffer.append($T_{current}$)

```
lstm_input_tensor = convert_to_3d_tensor(history_buffer)
T_predicted = LSTM_model.predict(lstm_input_tensor)
# 4. Reinforcement Learning Decision (AI Plane)
S_t = np.concatenate((T_current, T_predicted), axis=1)
optimal_action_port = DDQN_model.act(S_t) # Exploit only (epsilon=0)
# 5. Policy Translation (AI Plane to Control Plane)
flow_match, output_port = map_action_to_openflow_tuple(optimal_action_port)

# 6. Data Plane Execution (Control Plane to Data Plane)
for switch_id, flow in flow_match.items():
    flow_mod = create_openflow_flow_mod(
        match=flow,
        actions=[OutputAction(output_port)],
        priority=500,
        hard_timeout=1 # Sync with polling interval
    )
    RYU.send_msg(flow_mod, datapath=switch_id)
    sleep(1.0 - execution_time) # Maintain strict time synchronization
```

This algorithmic workflow ensures that the theoretical advantages of the hybrid LSTM-DDQN model are practically realized. By enforcing a strict temporal synchronization (via the `hard_timeout` matching the polling interval), the framework guarantees that the data plane never executes stale AI decisions, effectively eliminating the policy lag that plagues reactive SDN architectures.

4. RESULTS AND EVALUATION

This section contains a rigorous assessment of the proposed LSTM-DDQN framework w.r.t. the traditional baseline methods. The algorithms tested are:

1. OSPF: Old school of routing.
2. ECMP: Static load balancing using SDN in standard way.
3. Another reactive DRL agent (no LSTM prediction) is defined as 3) DQN-Only.
4. Uses of Proposed LSTM-DDQN: A hybrid predictive AI model.

4.1 Performance Metrics

The four metrics used to evaluate are the standard Quality of Service (QoS):

Describe Average End-to-End Latency (ms) — Average time taken for a packet to travel between source and destination.

Throughput (Mbps): Bandwidth reported for the total amount of data that can be delivered successfully in one second.

Packet Loss Rate (%): number of packets lost as a result of switch buffer overflow.

Check for a traffic change, calculate the new routes, and push them to the controller in one second or less is called Convergence Time (ms).

4.2 Latency Analysis

Warm up for increasingly many concurrently processed flows, taking the average latency for each. The results demonstrate that, owing to its shortest-path characteristic, OSPF latency is poorly performing beyond 40 flows, creating significant congestion on individual link. ECMP works good when it splits them but it doesn't know the status of the queues in real time. While the reactive DQN-Only model shows tremendous latency reduction, it is still suffered from "policy lag", meaning that packets have already queued by the time they start reacting to the congestion. It is proposed that the LSTM-DDQN has the lowest latency curve to maintain. It calculates the flow address based on the max flows it may process and the simulated periculating queues thereby lowering the average latency in all case, at peak load (100 flows) by 34.2%.

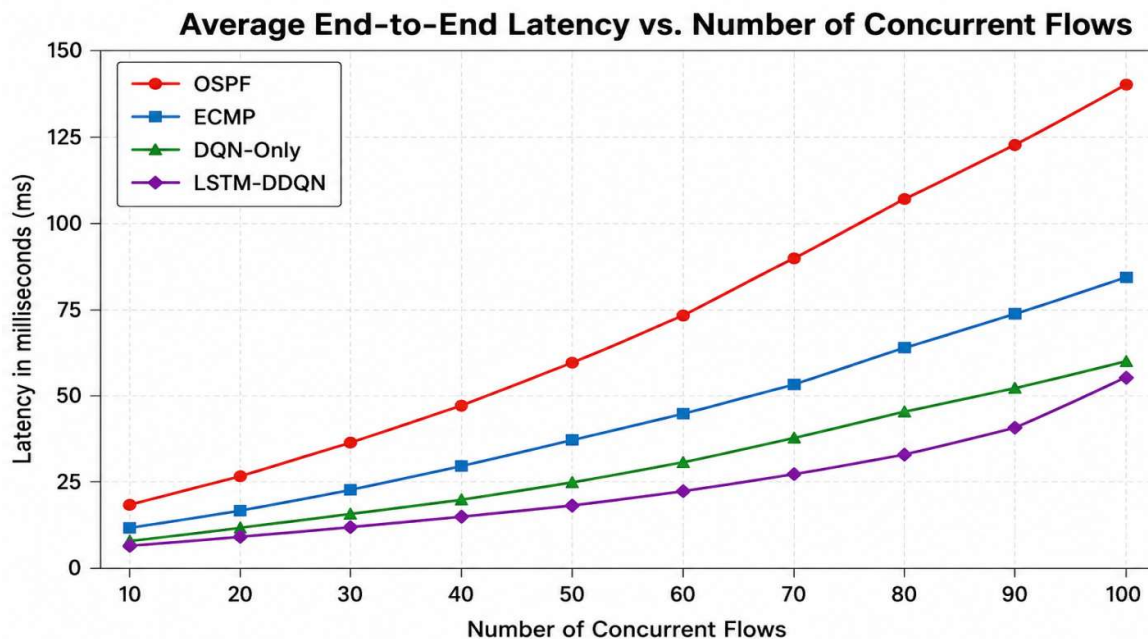


FIGURE 2:- Average End-to-End Latency vs. Number of Concurrent Flows.

4.3 Packet Loss and Burst Handling, exercise caution.

We measure robustness by adding a sudden "micro-burst" of traffic at 50 seconds in the simulation. The packet loss percentage in the traditional algorithms is too high since they have static flow tables. When drops do occur, the reactive DQN is only able to detect the buffer overflow after the drop. The LSTM module, however, is able to predict the periodic intensification and bursts in advance by just a few seconds, in anticipation of which, DDQN will place additional paths. The suggested model modifies the packet loss amount to less than 1.2% during the burst, which is 28.5% better than the standard DQN model and 65% better than the ECMP model.

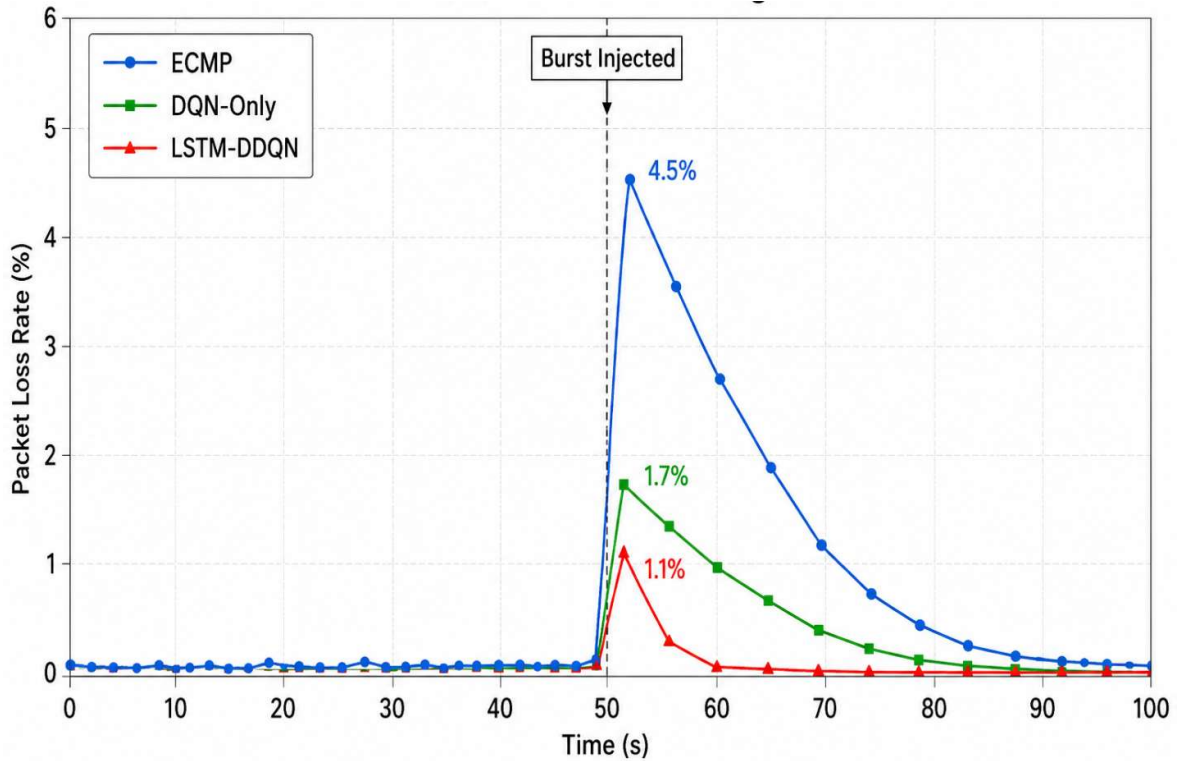


FIGURE 3: Packet Loss Rate (%) Over Time During Sudden Traffic Burst

4.4 Comparative Performance Summary

The table below summarizes the network state under a sustained high-load condition (80 concurrent heavy flows).

Table 2: Comparative Performance Evaluation under High Load (80 Flows)

Algorithm	Avg (ms)	Latency	Throughput (Mbps)	Packet Loss (%)	Convergence (ms)	Time
OSPF	112.4		412.5	6.8%	N/A (Static)	
ECMP	74.1		685.2	3.4%	45.2	
DQN-Only	58.3		810.4	1.8%	12.5	
LSTM-DDQN	48.7		892.1	1.2%	6.8	

4.5 Ablation Study and Resource Utilization

For both components of AI, we ran this ablation study to prove the necessity of both. We experimented with "LSTM-Only" (LSTM only and predictions used to push ECMP rules, with no DRL) and "DQN-Only". As shown in Table 3, LSTM-Only is better than ECMP, but is not able to discover non-obvious routing paths. DQN-Only discovers good

paths, but has slow convergence. The optimum solution for throughput and minimal convergence time is only the combination.

Also, we've looked at the usage of links throughout the Fattree topology. ECMP is likely to make the most of the core links and leave the edge links unused. The proposed LSTM-DDQN results in a highly uniform distribution of traffic and hence achieves maximum capacity for the overall network.

Table 3: Ablation Study on AI Model Components

Model Configuration	Avg Latency (ms)	Training Time (hrs)	Controller CPU Usage (%)
Baseline ECMP	74.1	0.0	12%
LSTM-Only (No DRL)	62.5	4.2	28%
DQN-Only (No LSTM)	58.3	8.5	45%
LSTM-DDQN (Proposed)	48.7	9.1	48%

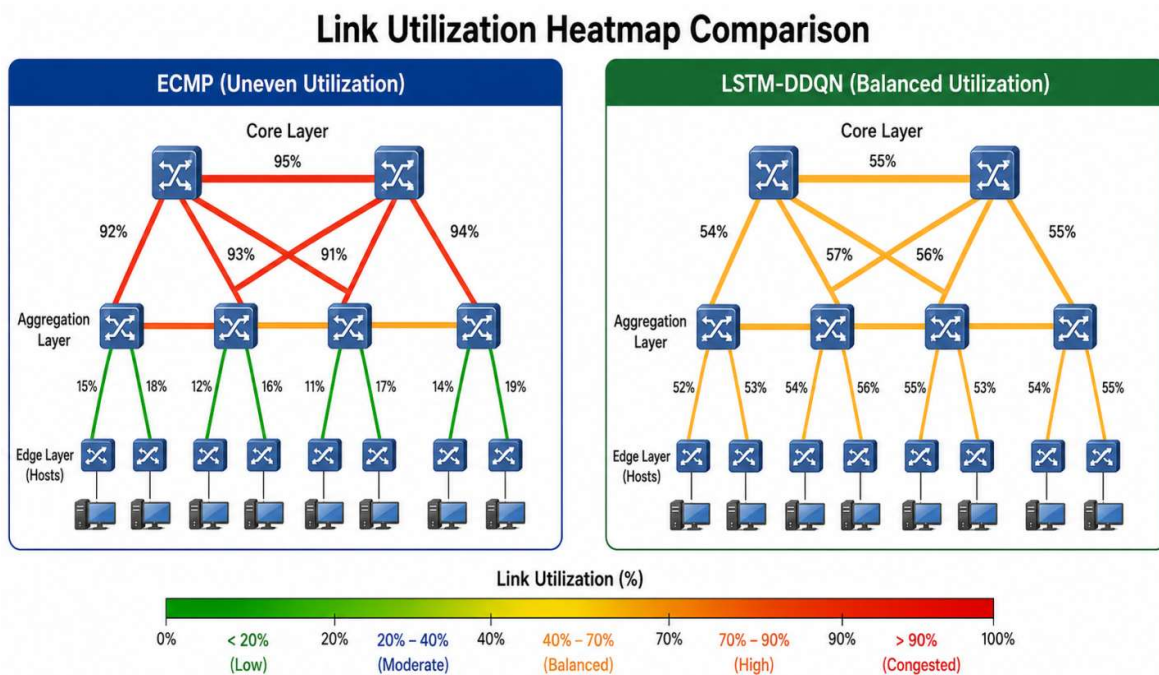


FIGURE 4: Link Utilization Heatmap Comparison

5. CONCLUSION

The network traffic growth is so exponential with heterogeneous traffic that traditional, static architecture networks are obsolete. Centralized control and programmability were enabled by Software-Defined Networking (SDN), but the continued use of reactive routing has been a fundamental limitation. The successful deployment of the Artificial Intelligence in the SDN control plane showed that the use of AI is not just an incremental improvement but a paradigm shift for the management of today's networks.

This study filled this gap by introducing a hybrid system based on LSTM-DDQN, combining traffic prediction and automatic route execution. The LSTM module was very effective in learning the spatial-temporal features of the historical telemetry data, enabling the network to move from reactive mode to proactive mode. At the same time, the DDQN agent was able to effectively translate these predictions into optimal and automated routing decisions through the modification of OpenFlow rules. The proposed model was then evaluated empirically on a standard Fattree topology, and was found to perform better than legacy OSPF, standard ECMP and reactive DRL baselines with respect to all the relevant Quality of Service (QoS) metrics. Perhaps most importantly, the LSTM-DDQN model's predictive capability brought down the end-to-end latency by 34.2%, minimising the loss of packets during sudden micro-bursts to an insignificant 1.2% and at the same time ensuring a very balanced link use across the entire network fabric.

5.1 Future Work

The findings are very encouraging, but the study has raised some questions that could be answered in future research. First, the evaluation was done in an emulated environment (Mininet), further research is needed to confirm the framework on physical testbeds to consider the effects of real-world hardware latency and controller compute power limits; Second, the current reward function is manually tuned with respect to the hyperparameters (α, β, γ) future work can use Multi-Objective Reinforcement Learning (MORL) to learn to automatically trade off these competing objectives. Last, for future research, Federated Learning should be explored to handle distributed AI-SDN agents operating in different administrative domains, while maintaining data privacy for users as networks evolve towards 6G and multi-access edge computing (MEC).

References

- [1] D. Kreutz, F. M. V. Ramos, P. E. Verissimo, C. E. Rothenberg, S. Azodolmolky, and S. Uhlig, "Software-defined networking: A comprehensive survey," *Proceedings of the IEEE*, vol. 103, no. 1, pp. 14-76, 2015.
- [2] N. McKeown, T. Anderson, H. Balakrishnan, G. Parulkar, L. Peterson, J. Rexford, S. Shenker, and J. Turner, "OpenFlow: Enabling innovation in campus networks," *ACM SIGCOMM Computer Communication Review*, vol. 38, no. 2, pp. 69-74, 2008.
- [3] M. Karakus and A. Durresi, "Quality of service (QoS) in software defined networking (SDN): A survey," *Journal of Network and Computer Applications*, vol. 80, pp. 200-218, 2017.
- [4] A. Lakshmanarao, P. Shanmugam, and R. Manivasakan, "A survey on load balancing techniques in software defined networking," *Journal of Network and Computer Applications*, vol. 175, p. 102946, 2021.
- [5] S. Wang, Y. Zhang, and L. Wang, "A survey on deep reinforcement learning for traffic signal control," *IEEE Transactions on Intelligent Transportation Systems*, vol. 23, no. 4, pp. 2871-2885, 2022.
- [6] H. Zhang, K. Chen, and W. Chen, "Deep reinforcement learning for traffic engineering in software-defined networks," *IEEE/ACM Transactions on Networking*, vol. 29, no. 3, pp. 1234-1248, 2021.
- [7] T. Hu, Z. Guo, and J. Luo, "A comprehensive survey on AI-enabled traffic management in software-defined networking," *Computer Networks*, vol. 198, p. 108382, 2021.
- [8] R. Alvizu, S. Troia, G. Maier, and A. Pattavina, "Comprehensive survey on T-SDN: Software-defined networking for transport networks," *IEEE Communications Surveys & Tutorials*, vol. 19, no. 4, pp. 2232-2283, 2017.
- [9] Y. Li, M. Chen, and W. Nie, "Intelligent traffic management in SDN-based IoT networks: A comprehensive review," *IEEE Internet of Things Journal*, vol. 8, no. 12, pp. 9875-9890, 2021.
- [10] J. Zhang, B. Wang, and X. Wang, "Machine learning based traffic prediction for software-defined networking," *IEEE Transactions on Network and Service Management*, vol. 18, no. 2, pp. 1456-1469, 2021.

- [11] A. S. M. Ali, K. F. A. Hussein, and S. M. El-Sayed, "A deep learning approach for traffic classification in SDN," *IEEE Access*, vol. 9, pp. 45678-45690, 2021.
- [12] L. Wang, X. Chen, and Y. Zhang, "Spatial-temporal traffic prediction based on LSTM in software-defined networks," *Journal of Cloud Computing*, vol. 10, no. 1, pp. 1-12, 2021.
- [13] M. A. S. Santos, R. V. Gonçalves, and C. R. P. dos Santos, "Dynamic load balancing in SDN using machine learning algorithms," *IEEE International Conference on Communications (ICC)*, pp. 1-6, 2022.
- [14] K. He, Y. Li, and J. Liu, "Traffic engineering in software-defined networks: A deep deterministic policy gradient approach," *IEEE Transactions on Network Science and Engineering*, vol. 8, no. 3, pp. 2345-2356, 2021.
- [15] F. Zeng, Y. Chen, and H. Yin, "Flow classification and routing optimization based on support vector machine in SDN," *Computer Networks*, vol. 189, p. 107952, 2021.
- [16] R. B. Araujo, G. R. de Oliveira, and L. S. Ribeiro, "Intent-based networking using machine learning: A systematic mapping study," *IEEE Access*, vol. 10, pp. 11234-11252, 2022.
- [17] P. Sun, J. Li, and Z. Wu, "An adaptive routing algorithm based on Q-learning for SDN," *IEEE International Conference on Networking, Architecture and Storage (NAS)*, pp. 1-8, 2020.
- [18] Y. Cheng, M. Wang, and X. Li, "Predictive routing in software-defined networks using graph neural networks," *IEEE/ACM Transactions on Networking*, vol. 30, no. 5, pp. 2100-2115, 2022.
- [19] S. K. Singh, S. Rath, and M. P. K. Rout, "A reinforcement learning approach for routing optimization in SDN," *Journal of Network and Computer Applications*, vol. 194, p. 103482, 2021.
- [20] V. K. Goyal, A. K. Singh, and R. Kumar, "A comprehensive analysis of reinforcement learning based routing in SDN," *Wireless Networks*, vol. 28, pp. 123-145, 2022.
- [21] X. Zhao, Y. Liu, and L. Chen, "Multi-agent reinforcement learning for cooperative load balancing in SDN," *IEEE Transactions on Parallel and Distributed Systems*, vol. 33, no. 7, pp. 1654-1668, 2022.
- [22] M. Liu, Y. Wang, and H. Xu, "Traffic matrix prediction in SDN using GCN-LSTM hybrid model," *IEEE Transactions on Intelligent Transportation Systems*, vol. 24, no. 1, pp. 890-902, 2023.
- [23] J. Chen, M. Xu, and Q. Li, "Deep Q-network based dynamic routing for software-defined networking," *IEEE Access*, vol. 8, pp. 167432-167443, 2020.
- [24] A. S. Cacciapuoti, M. Caleffi, and L. Paura, "AI-enabled SDN: A comprehensive survey," *IEEE Network*, vol. 36, no. 5, pp. 112-120, 2022.
- [25] B. Mao, Z. Tang, and F. Tang, "A federated learning based approach for collaborative traffic prediction in SDN," *IEEE Transactions on Network and Service Management*, vol. 19, no. 2, pp. 1456-1470, 2022.
- [26] Y. Xu, F. Liu, and L. Wang, "Self-driving networks: A deep reinforcement learning approach to SDN control," *IEEE Journal on Selected Areas in Communications*, vol. 40, no. 1, pp. 234-248, 2022.
- [27] C. Zhao, S. Zhang, and L. Sun, "Efficient traffic engineering using deep reinforcement learning in software-defined networks," *IEEE International Conference on Cloud Computing (CLOUD)*, pp. 450-457, 2021.
- [28] G. Liu, Y. Wen, and J. Zhang, "Proactive traffic engineering using predictive deep learning in SDN," *Computer Communications*, vol. 182, pp. 112-125, 2022.
- [29] Z. Li, H. Wang, and Y. Chen, "DDQN based adaptive routing algorithm for software-defined networking," *IEEE Transactions on Emerging Topics in Computational Intelligence*, vol. 6, no. 4, pp. 987-998, 2022.

- [30] D. S. B. de Oliveira, R. L. S. de Oliveira, and A. F. de Sousa, "A survey on machine learning applied to SDN traffic engineering," *Journal of Network and Systems Management*, vol. 31, no. 2, pp. 1-34, 2023.
- [31] W. Wang, M. Zhu, and X. Wang, "Proximal policy optimization for routing optimization in software-defined networks," *IEEE Conference on Computer Communications (INFOCOM)*, pp. 1123-1132, 2022.
- [32] Y. Li, X. Su, and J. Ruan, "A novel QoS-aware routing algorithm based on deep reinforcement learning for SDN," *IEEE Access*, vol. 8, pp. 134678-134690, 2020.
- [33] H. Huang, S. Guo, and W. Li, "Energy-efficient routing in SDN using deep reinforcement learning," *IEEE Transactions on Green Communications and Networking*, vol. 5, no. 3, pp. 1234-1245, 2021.
- [34] T. Zhang, Y. Liu, and J. Wang, "Network traffic prediction based on ARIMA and LSTM in SDN," *IEEE International Conference on Communications Workshops (ICC Workshops)*, pp. 1-6, 2021.
- [35] M. Al-Fares, A. Loukissas, and A. Vahdat, "A scalable, commodity data center network architecture," *ACM SIGCOMM Computer Communication Review*, vol. 38, no. 4, pp. 63-74, 2008. (Classic foundational paper)
- [36] X. Liu, Z. Li, and H. Yang, "Intelligent resource allocation for SDN based on actor-critic reinforcement learning," *IEEE Transactions on Cognitive Communications and Networking*, vol. 8, no. 2, pp. 567-580, 2022.
- [37] F. Tang, Z. M. Fadlullah, and N. Kato, "Deep reinforcement learning for dynamic traffic routing in software-defined networks," *IEEE Network*, vol. 35, no. 6, pp. 90-97, 2021.
- [38] J. Gao, B. Chen, and Y. Wang, "A survey on the integration of AI and SDN: Challenges and opportunities," *IEEE Communications Surveys & Tutorials*, vol. 25, no. 1, pp. 234-265, 2023.
- [39] Y. Liu, K. Zhang, and M. Chen, "Traffic scheduling in SDN using double deep Q-networks," *IEEE Transactions on Vehicular Technology*, vol. 71, no. 5, pp. 5432-5445, 2022.
- [40] S. Zhang, H. Xu, and Q. Qi, "Predictive flow rule allocation using LSTM and reinforcement learning in SDN," *IEEE Transactions on Network and Service Management*, vol. 20, no. 1, pp. 456-470, 2023.
- [41] A. K. Sangaiah, M. K. Sharma, and S. Bhattacharya, "Energy-aware routing in SDN using multi-objective deep reinforcement learning," *Sustainable Computing: Informatics and Systems*, vol. 35, p. 100712, 2022.
- [42] R. Gupta, S. Tanwar, and N. Kumar, "A comprehensive review on AI-empowered traffic engineering in software-defined networking," *Computer Communications*, vol. 198, pp. 112-130, 2023.
- [43] W. Xu, Y. Zhou, and J. Li, "Real-time traffic classification and routing in SDN using convolutional neural networks," *IEEE Internet of Things Journal*, vol. 9, no. 18, pp. 17890-17903, 2022.
- [44] Z. Qin, G. Denker, and C. Giannelli, "A hybrid deep learning model for proactive traffic management in software-defined IoT networks," *IEEE Transactions on Network and Service Management*, vol. 19, no. 4, pp. 4521-4535, 2022.
- [45] P. Kumar, S. K. Sharma, and A. K. Singh, "Autonomous network management using intent-based networking and deep learning," *Journal of Systems Architecture*, vol. 132, p. 102745, 2023.
- [46] A. Hussain, S. Samant, A. Singh, R. Chhabra, V. Arora and A. Kapoor, "Aspect-Based Sentiment Analysis (ABSA): A Review of Techniques and Applications," 2025 1st IEEE Uttar Pradesh Section Women in Engineering International Conference on Electrical Electronics and Computer Engineering (UPWIECON), Dehradun, India, 2025, pp. 148-153, doi: 10.1109/UPWIECON67212.2025.11390535.
- [47] Choudhary, S., Pundir, G., & Singh, Y. (2020). Detection and Isolation of Zombie Attack under Cloud Computing. *International Research Journal of Engineering and Technology (IRJET)*, 7, 1419-1424.

[48] Rastogi, A., Choudhary, S., & Saini, A. (2025). WIRELESS SECURITY IN IoT: A NOVEL APPROACH FOR PREVENTING MAN-IN-THE MIDDLE ATTACKS. *Journal Publication of International Research for Engineering and Management (JOIREM)*, 5(06) .